

Hybrides KI-System für Toxikokinetik

Dokumentation des Arbeitsablaufs und Anleitung zur Ausführung

Malaz al Mahdi

5. Mai 2026

Zusammenfassung

Die Arbeit kombiniert physik-/biologiebasierte Modelle (`httk` in R) mit maschinellen Verfahren in Python: Vorhersage der hepatischen intrinsic clearance Cl_{int} , PBTK-/IVIVE-Rechnungen, Erklärbarkeit (SHAP), neuronale Ansätze (Neural ODE, Graph Convolutional Network) sowie Risikoquotienten (AED, BER). Dieses Dokument erklärt jeden Arbeitsschritt, die erzeugten Daten und Ergebnisdateien sowie die konkreten Befehle zur Reproduktion unter Windows/PowerShell.

Inhaltsverzeichnis

1	Voraussetzungen und Installation	4
1.1	Software	4
1.2	Projekt-Repositories (überblick)	4
1.3	Installation (Kommandos)	4
2	Zielgrößen und Kurzdefinitionen	5
3	Schrittweise Pipeline (Stufen 1–14)	6
3.1	Stufe 1 – Datenextraktion aus httk	6
3.2	Stufe 2 – Random Forest & Gradient Boosting für Cl_{int} + externe Validierung . .	6
3.3	Stufe 3 – PBTK-Simulation	7
3.4	Stufe 4 – Reverse Dosimetry (Monte-Carlo AED)	7
3.5	Stufe 4b – AED-Analyse & Visualisierung (Python)	7
3.6	Stufe 5 – Vollständige RTK-Kette (~ 777 Chemikalien)	7
3.7	Stufe 6 – Neural ODE für konzentrationszeitliche Verläufe	8
3.8	Stufe 7 – Explainable AI (SHAP): global, BER und Ausreißeranalyse	8
3.9	Stufe 8 – Bayesian BER (Monte Carlo Dropout)	9
3.10	Stufe 9 – In-vivo-Validierung (R)	10
3.11	Stufe 10 – Graph Convolutional Network (Pilot, LOO-CV)	10
3.12	Stufe 11 – GCN & RF über alle ~ 777 Chemikalien + BER-Vergleich	11
4	Gemeinsamer Python-Code: scripts/utils.py	14
	Interpretationshinweise	14

1 Voraussetzungen und Installation

1.1 Software

- **R** (Version ≥ 4.2): Paket `httk` und typische Hilfspakete (`dplyr`, `ggplot2` je nach Skript).
- **Python** (≥ 3.10): Abhängigkeiten in `requirements.txt`.
- Für neuronale Komponenten und GCN zusätzlich: **PyTorch**, **rdkit**.

1.2 Projekt-Repositories (überblick)

`data/`

Eingabedaten und Zwischenstände (`.csv`).

`scripts/`

Alle R- und Python-Skripte; gemeinsamer Code in `scripts/Utils.py` (PK-Hilfen, Metriken, GCN-Klassen, SMILES-Abruf).

`results/`

Ausgaben: Tabellen (`.csv`), Metriken (`.txt`), Abbildungen (`.png`).

1.3 Installation (Kommandos)

In PowerShell im Projektstamm:

```
pip install -r requirements.txt
pip install torch rdkit
```

Installation von `httk` in R (Konsole oder per `Rscript`):

```
install.packages("httk", repos = "https://cloud.r-project.org/")
```

2 Zielgrößen und Kurzdefinitionen

- Cl_{int} (hepatische intrinsic clearance): Zielvariable der ML-Schritte ($\mu\text{L}/\text{min}/10^6$ Zellen nach `httk`-Konvention).
- **Deskriptoren für ML**: Molekulargewicht MW, Lipophilie $\log P$, freie Plasmafraktion F_{up} . (Frühere Version mit *Rblood2plasma* wurde zu Gunsten dieser drei reduziert.)
- **AED** (activity equivalent dose): aus IVIVE / Reverse-Toxikokinetik und AC50-/Assayinformationen ableitbar.
- **BER** (bioactivity exposure ratio): Verhältnis bioaktiver „äquivalenter“ Dosis zur geschätzten Exposition (z. B. SEEM), je nach Schritt verschiedene Skalierung.

3 Schrittweise Pipeline (Stufen 1–14)

Die Reihenfolge entspricht der empfohlenen Abhängigkeiten zwischen Skripten. Das gesamte Shell-Orchestrierungsskript ist `scripts/run_pipeline.ps1`.

3.1 Stufe 1 – Datenextraktion aus `httk`

Skript: `01_extract_httk_data.R`

Zweck: Pilotchemikalien aus `httk` auswählen und als Tabelle sichern (MW, logP, F_{up} , gemessene Cl_{int} , ggf. weitere Felder).

Typische Outputs:

- `data/pilot_chemicals_full.csv` – Vollständige Kennzahlen,
- `data/pilot_chemicals_masked.csv` – Variante zum Training mit maskiertem Clint.

Ausführung:

```
cd scripts
Rscript 01_extract_httk_data.R
```

3.2 Stufe 2 – Random Forest & Gradient Boosting für Cl_{int} + externe Validierung

Skript: `02_rf_predict_clint.py`

Zweck: Feature Engineering (9 Features aus MW, logP, F_{up}), Training von RF und GB mit Leave-One-Out-CV auf den Pilotchemikalien; Auswahl des besseren Modells; Imputation für fehlende Clint-Werte. Im Anschluss direkte externe Validierung: das trainierte Modell wird auf alle ~ 777 `httk`-Chemikalien mit gemessenem Cl_{int} angewendet und Metriken (RMSE, GMFE, Fold-Errors) gegen Literaturwerte (Wetmore 2012) berechnet. Damit entfällt ein separater Validierungsschritt.

Nutzt: `utils.engineer_features`, `utils.compute_metrics`, gemeinsame Pfade aus `utils.py`.

Outputs:

- `data/rf_clint_predictions.csv` – LOO-CV Vorhersagen,
- `data/pilot_chemicals_imputed.csv` – RF-imputierter Datensatz,
- `results/rf_loo_cv_metrics.txt` – Interne LOO-CV Metriken,
- `results/clint_validation_metrics.csv` – Intern + extern im Vergleich,
- `results/clint_validation_external.csv` – Vollständige Vorhersagetabelle (777),
- `results/clint_validation_scatter.png` – Log-log Scatter (4 Panels).

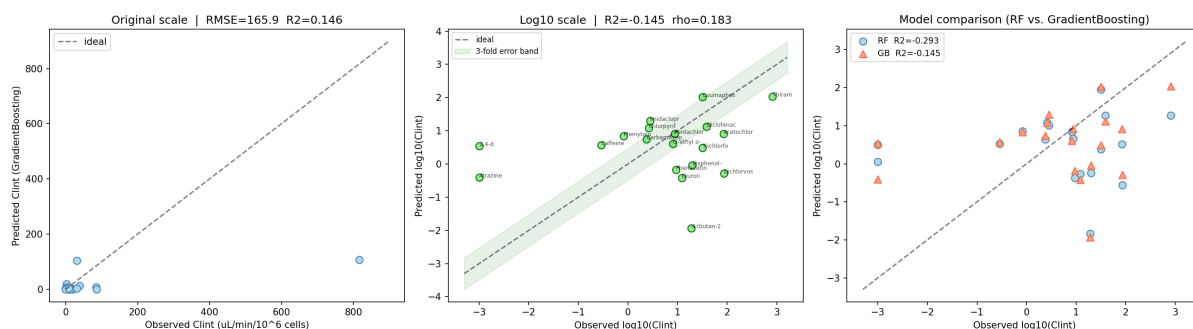


Abbildung 1: Leave-One-Out-Cross-Validation: Vorhergesagter vs. gemessener Clint-Wert

```
python 02_rf_predict_clint.py
```

3.3 Stufe 3 – PBTK-Simulation

Skript: 03_httk_pbtk_simulation.R

Zweck: Vergleich von Simulationen mit httk-nativem Clint gegen RF-imputiertem Clint (z. B. Konzentrations-Zeit-Kurven, AUC, Css).

Output typisch: results/pbtk_comparison.csv, Kurvengrafiken falls implementiert.

```
Rscript 03_httk_pbtk_simulation.R
```

3.4 Stufe 4 – Reverse Dosimetry (Monte-Carlo AED)

Skript: 04_reverse_dosimetry.R

Zweck: IVIVE/Reverse TK: Von ToxCast-ähnlichen Bioaktivitätsdaten auf orale Äquivalentdosen

(Monte-Carlo-Unsicherheit über httk-Populationsvariabilität).

Output: z. B. AED-Stichproben und Zusammenfassungen in results/.

```
Rscript 04_reverse_dosimetry.R
```

3.5 Stufe 4b – AED-Analyse & Visualisierung (Python)

Skript: 04b_aed_analysis.py

Zweck: Paarvergleiche und Plots zur AED-Analyse (native vs. ML-skaliertes Pfad, je nach Datenlage).

```
python 04b_aed_analysis.py
```

3.6 Stufe 5 – Vollständige RTK-Kette (~777 Chemikalien)

Skript: 05_full_rtk_aed_ber.R

Zweck: Skalierung auf die httk-parameterisierbaren Substanzen, Koppelung von Expositionsschätzungen (NHANES/SEEM, je nach httk-/Projektlogik), Berechnung von AED und BER über viele Chemikalien.

Zentraloutput: results/aed_ber_full.csv, Zusammenfassungen wie aed_ber_summary.csv.

```
Rscript 05_full_rtk_aed_ber.R
```

3.7 Stufe 6 – Neural ODE für konzentrationszeitliche Verläufe

Skript: 06_neural_ode_tk.py

Zweck: Lernen eines dynamischen Systems für $C(t)$ mit neuronal parametrisiertem Feld (ODE-Netzwerk), Training oft gegen analytische 1-Kompartiment-Vorläufigkeit.

Voraussetzung: PyTorch.

Output: u. a. results/neural_ode_metrics.csv,

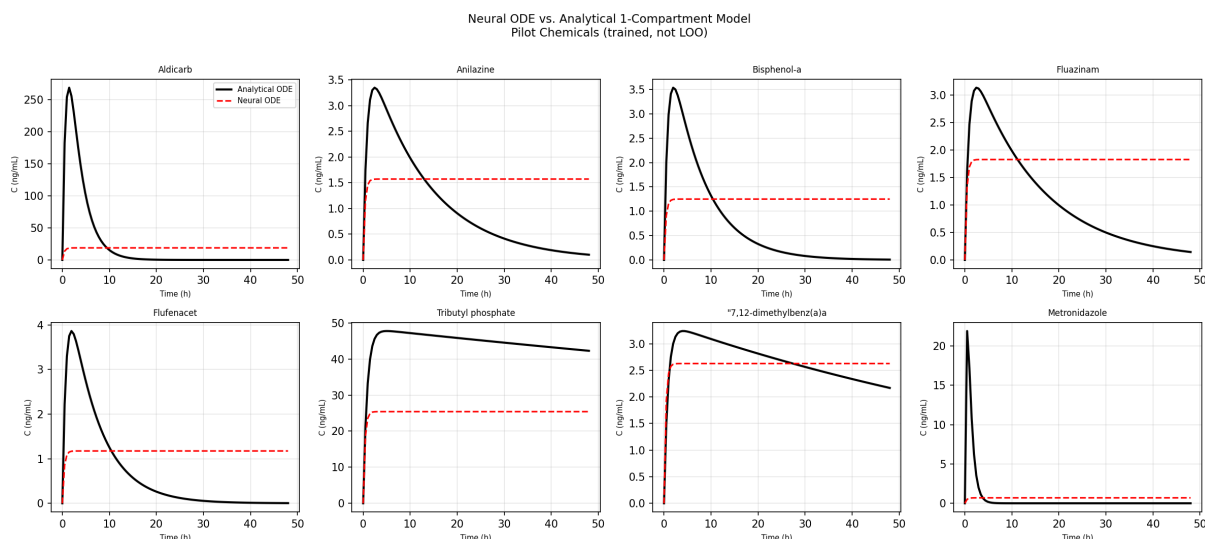


Abbildung 2: Neural ODE: Vorhergesagte vs. analytische $C(t)$ -Kurven

```
python 06_neural_ode_tk.py
```

3.8 Stufe 7 – Explainable AI (SHAP): global, BER und Ausreißeranalyse

Skript: 07_xai_shap_analysis.py

Zweck: Vollständige SHAP-Analyse in drei Sektionen, zusammengeführt aus dem früheren separaten Ausreißer-Skript:

- **Section A:** TreeExplainer für das RF-Clint-Modell – globale Wichtigkeit, Beeswarm, Dependence Plots (logP, Fup, MW).
- **Section B:** SHAP für ein BER-erklärendes Modell – warum wird eine Chemikalie als hoch-priorisiert eingestuft?
- **Section C:** Gezielte Ausreißeranalyse (9-Feature-Modell, alle 777 Chemikalien) – Waterfall-Plots für die 5 am schlechtesten vorhergesagten Stoffe sowie ein Vergleich zwischen Ausreißern (Tacrin, Phenylparaben, Acibenzolar) und gut vorhergesagten Referenzchemikalien. Erklärt mechanistisch, warum reine physikochemische Deskriptoren bei enzymespezifischem Metabolismus versagen.

Voraussetzung: Paket shap.

Outputs: u. a. results/shap_rf_values.csv, results/shap_outlier_values.csv, results/shap_outlier_v

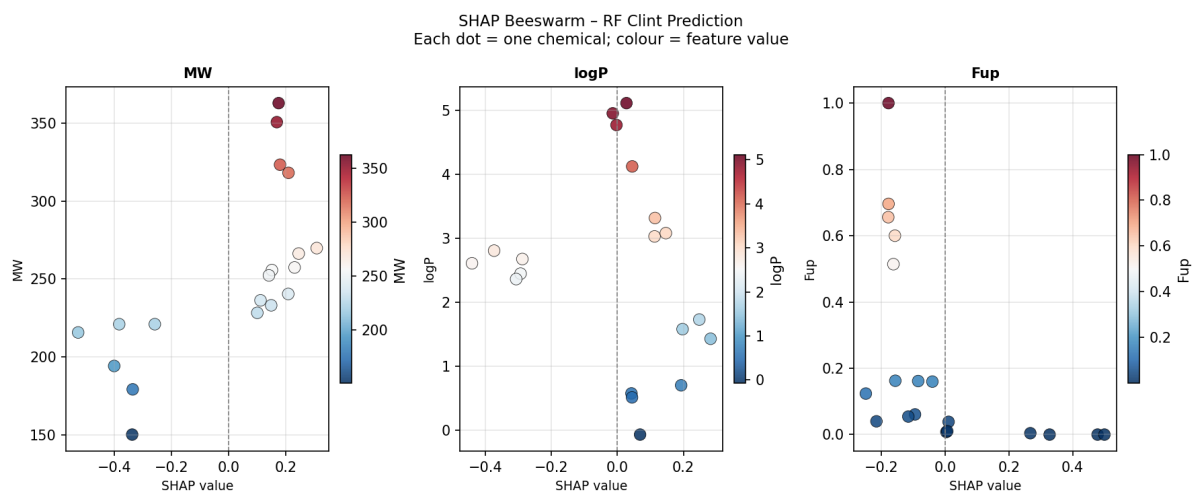


Abbildung 3: SHAP Beeswarm: globale Feature-Wichtigkeit für das RF-Clint-Modell

```
python 07_xai_shap_analysis.py
```

3.9 Stufe 8 – Bayesian BER (Monte Carlo Dropout)

Skript: 08_bayesian_ber.py

Zweck: Kleines Bayesian MLP für $\log_{10} Cl_{int}$; bei Inferenz Dropout aktiv → posterior-ähnliche Stichproben; Propagation zu AED und BER (bei gültigen Expositionsdaten).

Nutzt: `utils.clint_uL_to_cl_h` konsistent zur PK-Schicht.

Outputs: `results/bayesian_ber.csv`,

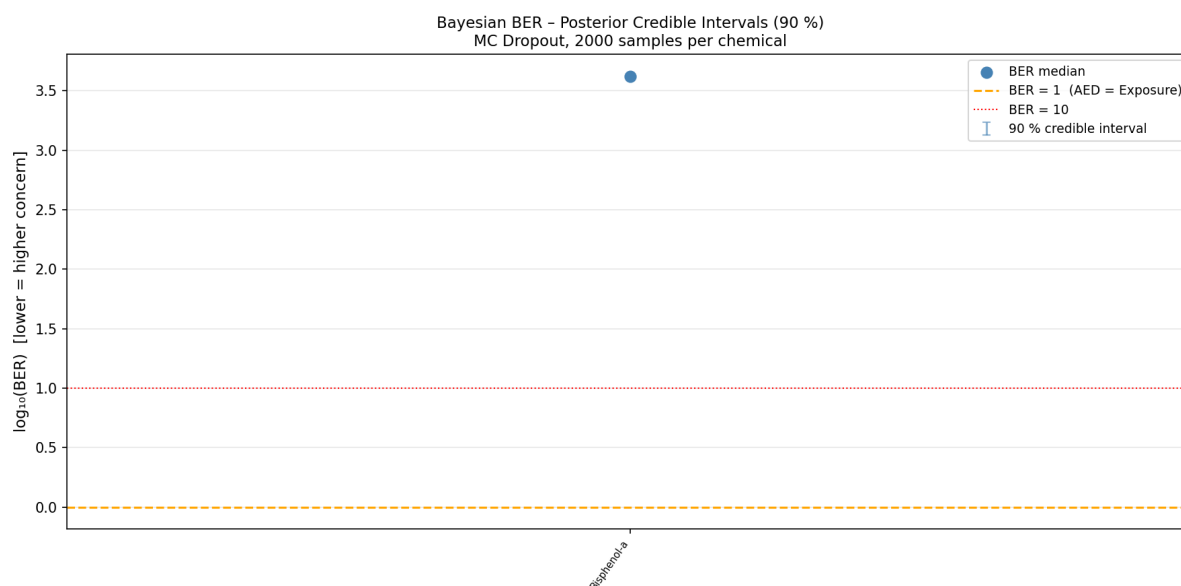


Abbildung 4: Bayesianische BER-Glaubwürdigkeitsbänder (90%-KI, MC Dropout)

rior BER Distributions – Top-5 Highest-Concern Chem

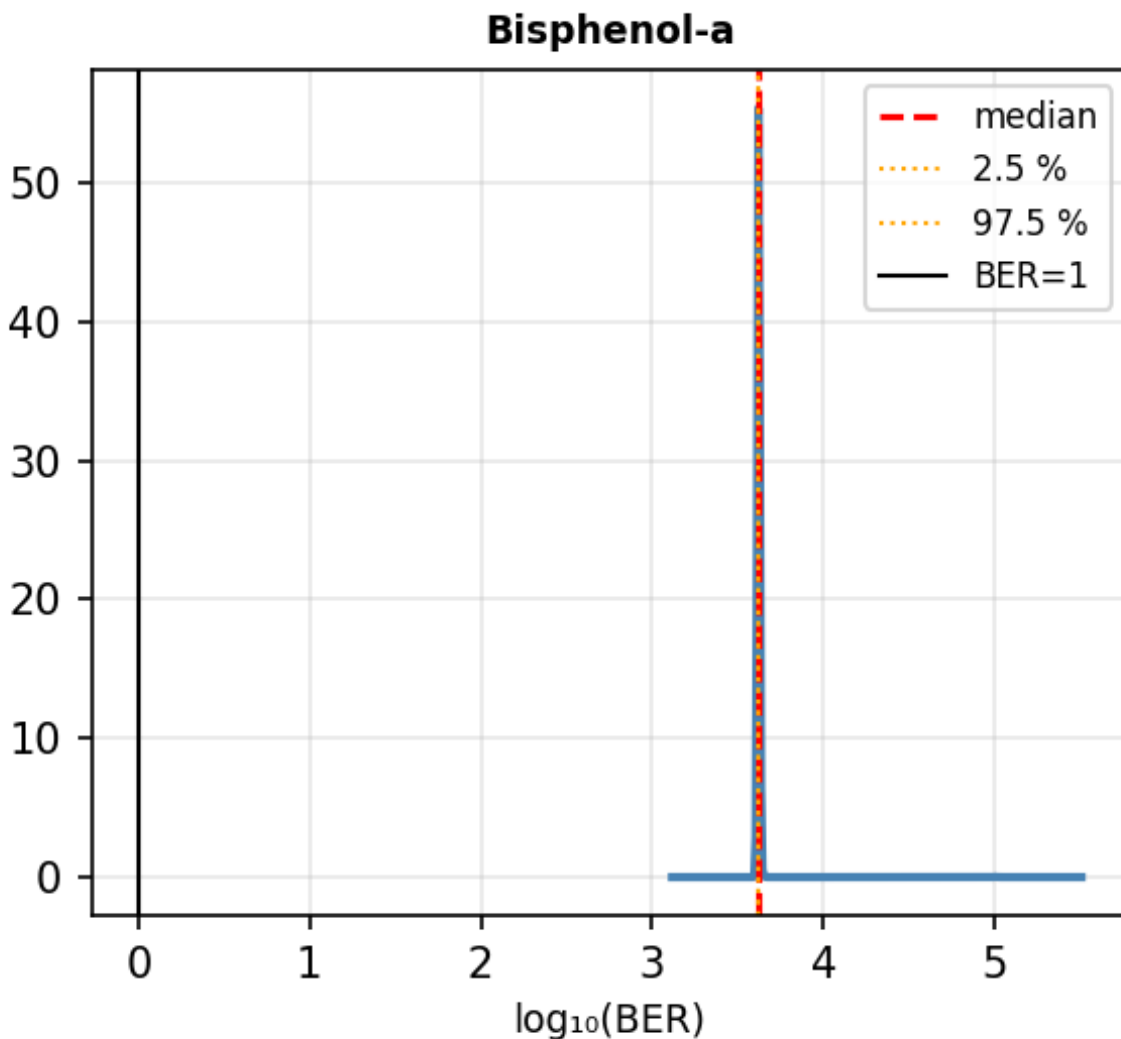


Abbildung 5: Posteriori-Verteilung BER: Top-5 Chemikalien

```
python 08_bayesian_ber.py
```

3.10 Stufe 9 – In-vivo-Validierung (R)

Skript: 09_invivo_validation.R

Zweck: Abgleich httk-/Modellprognosen mit publizierten In-vivo-Messungen (z. B. Css, Wetmore 2012 / verwandte Quellen aus httk).

Ausgabe typisch: results/invivo_validation_*.csv/png, falls erfolgreiche Ausführung.

```
Rscript 09_invivo_validation.R
```

3.11 Stufe 10 – Graph Convolutional Network (Pilot, LOO-CV)

Skript: 11_gcn_clint.py

Zweck: Molekulare Graphen aus SMILES (RDKit), GCN lernt nur aus den Pilotmessungen; Auswertung per LOO-CV. Kann je nach Daten und Epoch-Länge mehrere Minuten in Anspruch nehmen.

Abhängigkeiten: rdkit, torch; SMILES-Recherche z. B. über PubChem.

Klassen: GCN-Architektur und Trainingsroutine aus `utils.py` (`MolGCN`, `train_gcn`, `predict_gcn`).

Output: `results/gcn_loo_cv_*.txt/png`,

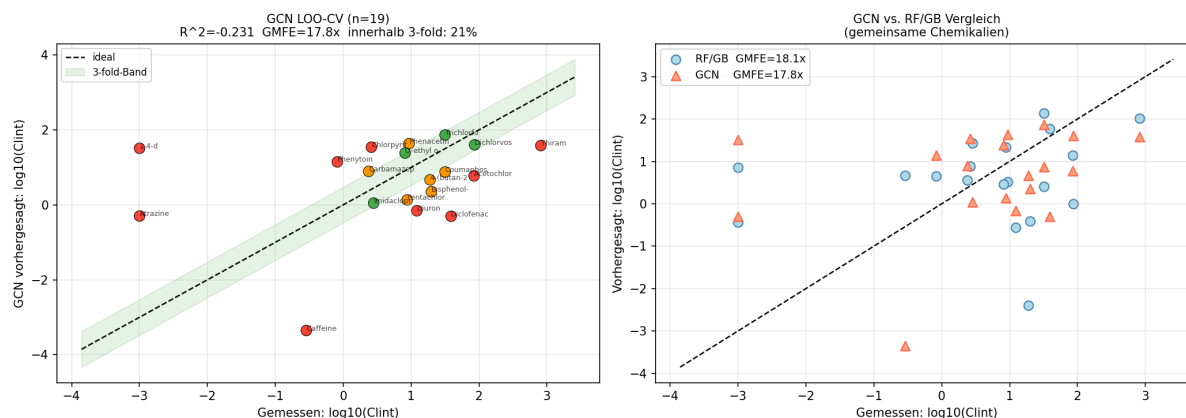


Abbildung 6: GCN LOO-CV: Vorhergesagter vs. gemessener Clint auf den 19 Pilotchemikalien

`data/pilot_chemicals_gcn.csv`.

```
python 11_gcn_clint.py
```

3.12 Stufe 11 – GCN & RF über alle ~777 Chemikalien + BER-Vergleich

Skript: `13_gcn_all777.py`

Zweck: SMILES-Cache (`data/smiles_cache_777.csv`), GCN trainiert nur auf Pilotdaten, Vorhersage für alle mit gültigem Graphen; paralleler RF/GB-Pfad ohne SMILES für alle Chemikalien; Literatur-Clint dort, wo vorhanden → externe Güteplots. Im Anschluss direkte BER-Berechnung (früher separates Skript `14_ber_all777.py`): AED und BER werden aus allen drei Cl_{int} -Quellen (httk-Literatur, GCN, RF) mit `utils.calc_aed` und `utils.calc_ber` berechnet und verglichen – kein zwischenzeitlicher CSV-Reload nötig.

Outputs: `results/gcn_777_predictions.csv`, `results/gcn_777_metrics.txt`, `results/ber_all777.csv`, `results/ber_all777_metrics.txt`,

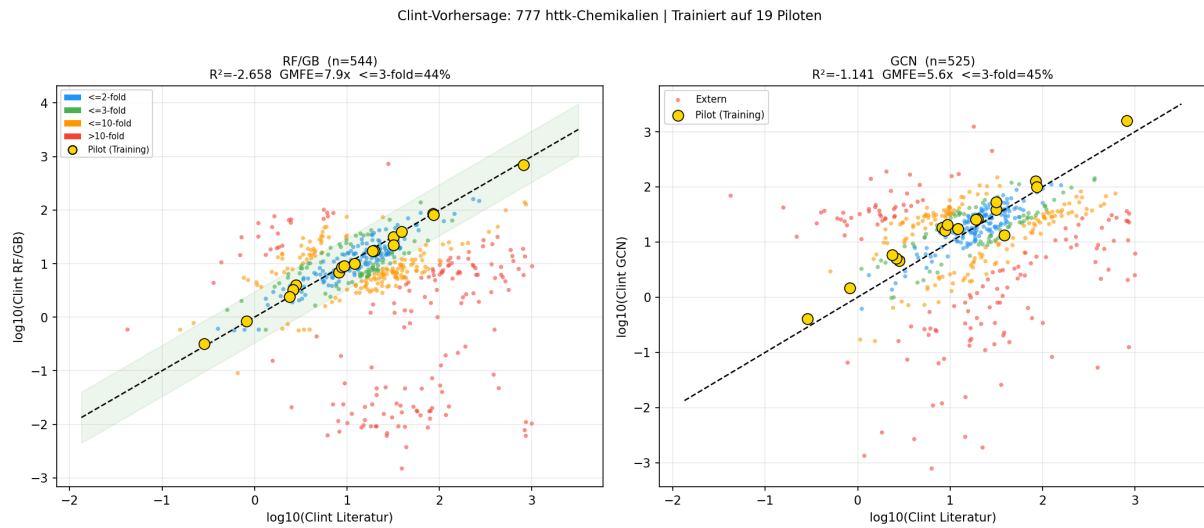


Abbildung 7: GCN + RF/GB Clint-Vorhersage: alle ~777 htk-Chemikalien

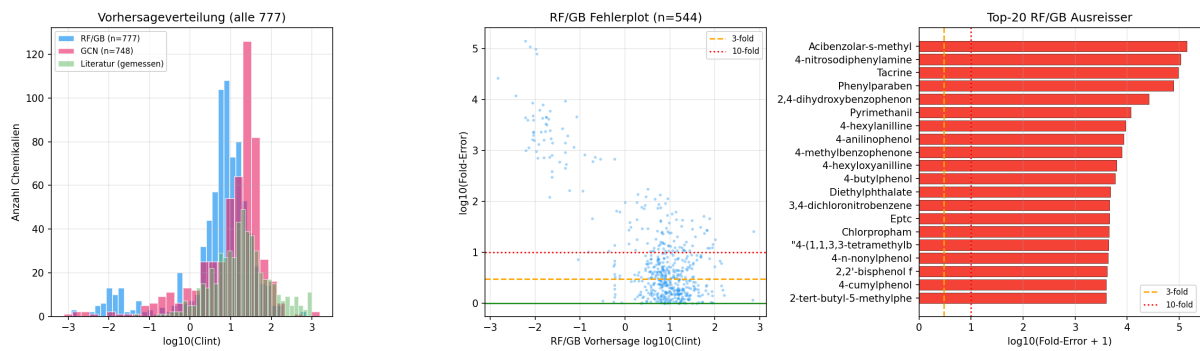


Abbildung 8: Clint-Verteilung: htk-Literatur vs. GCN vs. RF/GB

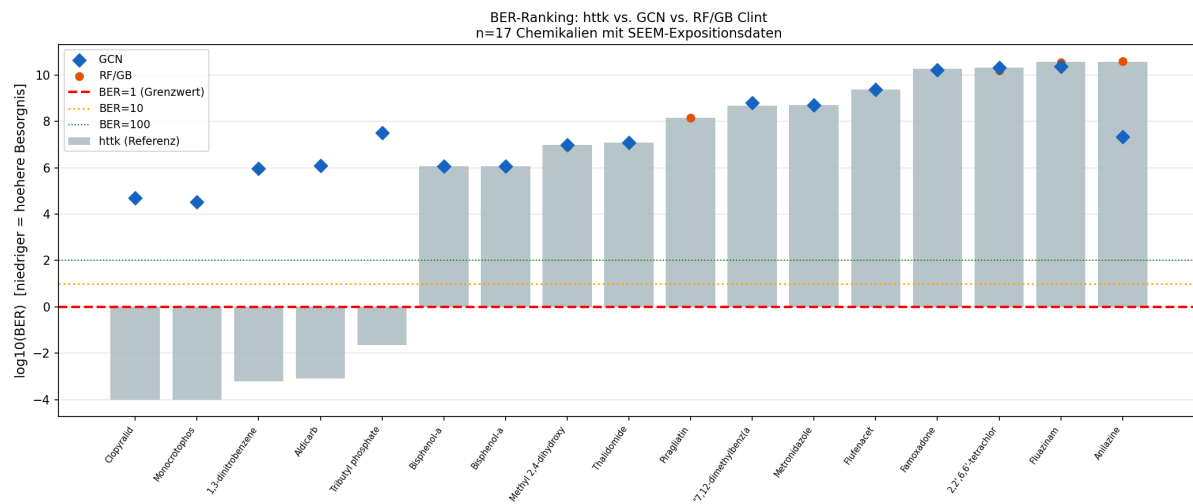


Abbildung 9: BER-Ranking: htk vs. GCN vs. RF/GB Clint für alle Chemikalien mit SEEM-Expositionsdaten

```
python 13_gcn_all1777.py
```

4 Gemeinsamer Python-Code: scripts/utils.py

Die Datei zentralisiert:

- Konstanten zur hepatischen Extraktion (Blutfluss Q_H , Lebergewichtssektor, Hepatozytenzahl), Funktion `clint_uL_to_cl_h`, `calc_aed`, `calc_ber`.
- Identisches Feature Engineering wie in Stufe 2 (`engineer_features`).
- Einheitliche Metriken (`compute_metrics`) für logarithmierte Clint-Vergleiche.
- SMILES über PubChem und CIR sowie Graphaufbau `mol_graph` sowie GCN-Training.

So bleiben die Skripte 02, 08, 13, 14 ohne Mehrfachimplementierungen wartbar.

Interpretationshinweise (wissenschaftlich)

1. **Kleinstichprobe Pilot ($n \approx 19$):** Stark generalisierende Modelle können negative R^2 auf Rohskala oder extern zeigen — deshalb wurden GMFE und Fold-Toleranz ($\times 2/3/10$) betont.
2. **GCN ohne Massendaten zur Metabolismusenzyomatik** kann strukturelle Informationen liefern, ersetzt aber keine Enzyme: Ausreißer bleiben plausibel.
3. **BER** erfordert Exposition: Ohne gültiges SEEM keine vollständige BER-Zeile, obwohl AED häufiger berechenbar bleibt.

Literatur

- [1] Pearce & Wambaugh *et al.*: R-Paket `httk`, EPA/CompTox.
- [2] Wetmore B.A. *et al.* (2012) *Toxicol. Sci.*
- [3] Lundberg & Lee (2017) SHAP — Modellinterpretation (TreeExplainer bei Bäumen).